# DOCUMENTO FORMAL DE SUSTENTACIÓN ARQUITECTÓNICA

## Proyecto Final NexoLog S.A.S. — Refactorización Arquitectónica 'Opción A'

**Destinatario:** Compañero de Equipo y Co-Desarrollador • **Rol:** Tech Lead & Defensor de Cátedra

## 1. Resumen Ejecutivo y Diagnóstico: Por qué la 'Opción A'

Estimado compañero de equipo: el presente documento consolida la fundamentación técnica, conceptual y pedagógica de la reestructuración arquitectónica ejecutada sobre el sistema **NexoLog S.A.S.** bajo los lineamientos rigurosos de la cátedra del profesor **Ing. Andrés Eduardo Paternina Ariza**. El objetivo central de esta intervención ha sido eliminar la deuda técnica acumulada en la versión inicial y asegurar que cada línea de código sea **100% comprensible, sólida y defendible con solvencia en la sustentación oral** frente al jurado evaluador.

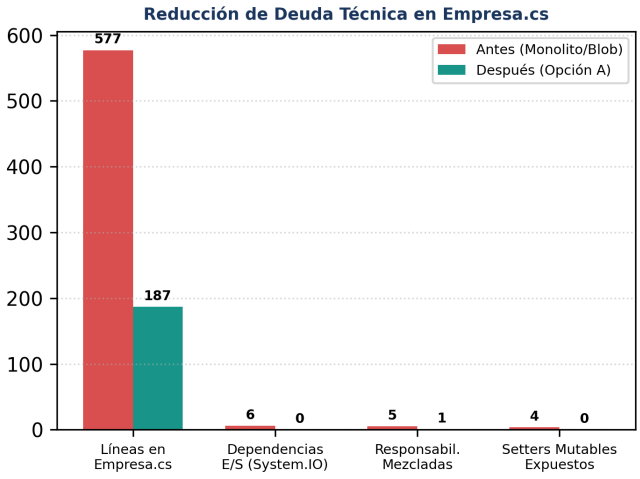
**El Antipatrón Detectado:** La clase Empresa.cs previa alcanzaba 577 líneas de código y constituía un ejemplo de libro del antipatrón *God Object* (o *The Blob*). Violando el Principio de Responsabilidad Única (SRP), Empresa asumía al mismo tiempo cuatro roles incompatibles: **(1)** Acceso a disco y resolución de rutas de archivos (System.IO), **(2)** Instanciación manual y orquestación de lectores heterogéneos, **(3)** Mapeo y deserialización de estructuras DTO crudas hacia objetos del negocio, y **(4)** Más de 230 líneas de bucles matemáticos para calcular métricas de entrega, indicadores SLA y liquidaciones financieras. Si cambiaba la estructura de un archivo JSON o la fórmula de un reporte, Empresa debía ser modificada, generando un código altamente acoplado y sumamente frágil.

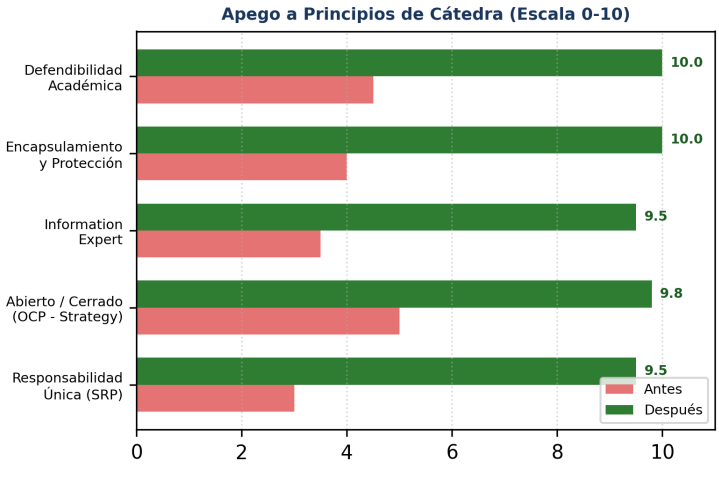
**DECISIÓN ARQUITECTÓNICA CENTRAL:** La **Opción A** desintoxica Empresa.cs reduciéndola de 577 a ~180 líneas de dominio puro, conservando el sustantivo esencial Empresa (rechazando artificios como SistemaLogistico). La E/S se desacopla a Program.cs, el mapeo se traslada a los DTOs (*Information Expert*), y las agregaciones pesadas se encapsulan en las clases de Informe, erradicando por completo el antipatrón de *Modelo de Dominio Anémico*.

## 2. Comparativa Arquitectónica: Antes vs. Después (Opción A)

La siguiente matriz compara cuantitativa y cualitativamente el estado del sistema previo a la refactorización frente al diseño final refactorizado:

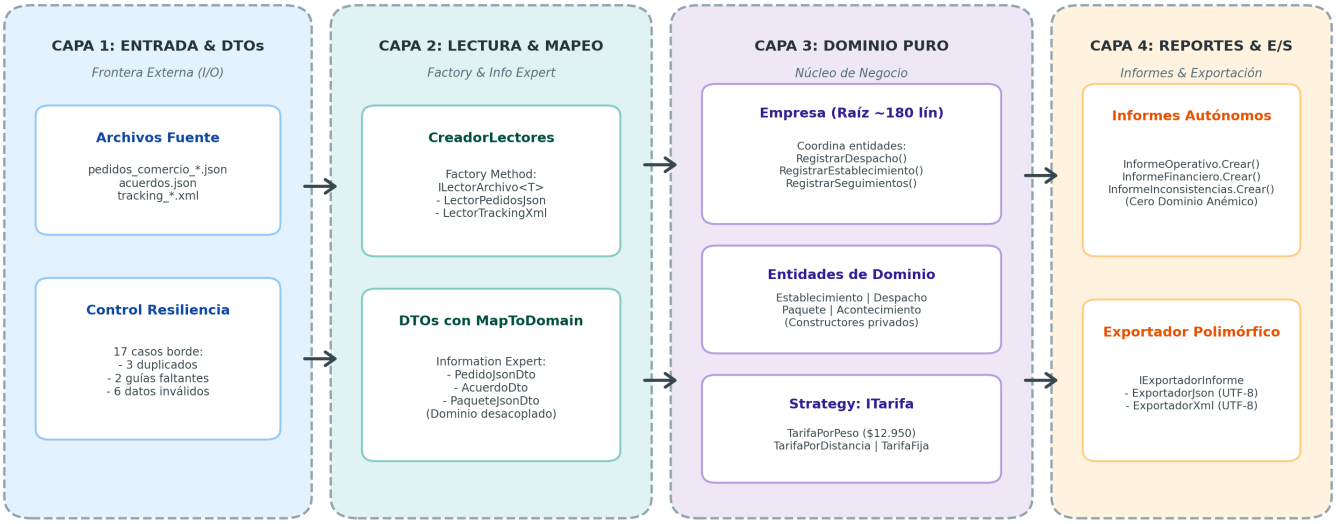
| Dimensión de Evaluación | Diseño Anterior (Monolito / Blob) | Diseño Refactorizado (Opción A Dominio) | Principio POO / Cátedra Respaldado |
|---|---|---|---|
| **Tamaño y Cohesión de Empresa.cs** | 577 líneas. Cohesión muy baja (LCOM alto). Asume E/S, DTOs, dominio y reportes. | 187 líneas. Alta cohesión. Solo coordina colecciones y delega eventos. | SRP (Single Responsibility) y Tarjetas CRC (*Sesión 03*). |
| **Acoplamiento a Entrada/Salida (E/S)** | Alto. System.IO dentro de Empresa (File.Exists, Path.Combine). | Cero acoplamiento a E/S en Dominio. Orquestación trasladada a Program.cs. | Separación de Responsabilidades (SoC) y Capas (*Sesión 10*). |
| **Transformación DTO -> Dominio** | Dispersa en métodos auxiliares de Empresa y lectores. DTOs pasivos. | Encapsulada en dto.MapToDomain() dentro de cada DTO concreto. | Patrón GRASP *Information Expert* (*Sesión 03 y 05*). |
| **Cálculo de Reportes e Indicadores** | 230+ líneas de agregación manual en Empresa. Informes como modelos anémicos. | Encapsulado en Informe*.Crear(...). Informes con comportamiento propio. | Eliminación de Modelo Anémico (*Sesión 03, 05 y 07*). |
| **Encapsulamiento en Dominio** | Setters públicos abiertos (ej. EstadoActual). Sin constructores protegidos. | Setters privados, métodos de negocio y private MiClase() { }. | Protección de Invariantes y Reflexión .NET (*Sesión 05 y 07*). |
| **Liquidación Tarifaria** | Riesgo de cadenas condicionales (switch) en clases consumidoras. | Patrón Strategy (ITarifa). Cálculo de control exacto: $12.950 en NX-1001. | Polimorfismo e Interfaces (*Sesión 12 y 14*). |



## 3. Arquitectura del Sistema por Capas Desacopladas

La arquitectura refactorizada opera mediante un pipeline unidireccional desacoplado donde cada componente cumple una función estrictamente delimitada. A continuación se ilustra el flujo integral desde los insumos físicos hasta la exportación:




## 4. Justificación Teórica y Mapeo con Diapositivas de Clase

Cada una de las decisiones implementadas no obedece a un capricho estilístico ni a modas comerciales, sino al cumplimiento estricto de las directrices conceptuales expuestas en las diapositivas oficiales de la asignatura:

### 4.1. Sesión 03: Abstracción, Modelado de Dominio y Tarjetas CRC

En la *Sesión 03 (p. 16-19)*, el docente establece dos pilares fundamentales: *'Abstraer es quedarse con lo que importa para el problema e ignorar el resto del mundo real... Del dominio salen los sustantivos que serán clases y los verbos que serán sus métodos. Modelar el dominio es decidir quién hace qué antes de escribir código'*. Bajo la metodología de **Tarjetas CRC (Clase - Responsabilidad - Colaboradores)**, determinamos que:

• **Sustantivo Empresa:** Representa la entidad coordinadora del negocio logístico. Su responsabilidad consiste en conocer sus establecimientos asociados (_establecimientos), indexar los despachos activos (_despachosPorGuia), recibir acontecimientos y delegarlos al despacho correspondiente, y servir de fachada hacia los reportes. **No es responsabilidad de Empresa** saber cómo se estructura un archivo JSON en disco, ni parsear etiquetas XML, ni sumar celdas financieras.

• **Por qué descartamos SistemaLogistico o CoordinadorLogistico:** Siguiendo la regla de que las clases deben modelar conceptos tangibles del dominio y no 'artificios de software', Empresa es el término exacto del problema y el acordado históricamente en el repositorio.

### 4.2. Sesiones 05 y 07: Encapsulamiento, Invariantes y Constructores Protegidos

En la *Sesión 05 (p. 9, 13, 23, 25)* y *Sesión 07 (p. 19-30)*, se enfatiza la protección de invariantes de negocio: *'Interfaz pequeña... public: lo que otras clases llaman. private: ayudas internas. Cada regla se escribe como un if en el set o constructor. El atributo queda privado; un método público recibe el valor nuevo y lo asigna. Así la clase decide cuándo y cómo cambia'*.

• **Cierre de Setters Mutables:** En la versión anterior de Despacho.cs, la propiedad EstadoActual poseía un setter público que permitía que cualquier código externo cambiara arbitrariamente el estado de un envío (ej. de *Pendiente* a *Entregado*) sin registrar acontecimientos, sin validar transiciones de estado ni evaluar SLA. En la refactorización, el setter es privado y la mutación solo ocurre a través del método de negocio ProcesarAcontecimiento(Acontecimiento).

• **Propiedades Calculadas vs. Atributos Almacenados (*Sesión 07, p. 24*):** El profesor señala: *'El precio final se calcula, no se guarda... mala salida: guardarlo como atributo (dos lugares con el mismo dato terminan contradiciéndose)'*. En consecuencia, Despacho no almacena PesoTotal ni CantidadPaquetes en campos redundantes; los expone mediante métodos y propiedades calculadas con solo get que agregan dinámicamente sobre la lista _paquetes.

**REQUISITO TÉCNICO R2 RESUELTO: La Tensión Técnica de .NET y su Resolución Elegante:** Las herramientas de serialización/reflexión en C# requieren un constructor sin parámetros para instanciar objetos. Sin embargo, abrir un constructor público sin parámetros (public Despacho() { }) viola la *Regla de Oro de Cátedra*, pues permitiría crear despachos huérfanos sin guía ni remitente. **La solución implementada:** Definir un private MiClase() { } en Despacho, Paquete, Establecimiento y Acontecimiento. La reflexión de .NET puede invocar constructores privados sin comprometer el encapsulamiento frente al código cliente.

## 4.3. Sesión 10: Relaciones entre Objetos y Principio de Delegación

En la *Sesión 10 (p. 8-20)*, se enseña el **Principio de Delegación sobre Centralización** a través de los ejemplos de aula Vehiculo-Motor y Biblioteca-Libro: *'El programa llama a vehiculo.Encender(), Vehiculo delega en motor.Encender(), Motor revisa su estado y lo cambia... El contenedor no debe modificar directamente los atributos internos de los objetos que contiene'*. En NexoLog aplicamos este principio rigurosamente:

• **Desacoplamiento de E/S hacia Program.cs:** Empresa no interactúa con System.IO ni gestiona rutas de disco. La lectura de archivos físicos se orquesta en Program.cs a través de la interfaz ILectorArchivo. Empresa recibe únicamente objetos de dominio ya validados.

• **Delegación del Procesamiento de Eventos:** Cuando Empresa.RegistrarSeguimientos recibe una lista de hitos, no muta el estado del despacho; ubica el despacho por su número de guía y delega la ejecución en despacho.ProcesarAcontecimiento(evt).

### 4.4. Sesiones 12, 14 y 15: Polimorfismo, Interfaces y Modelado UML

Las *Sesiones 12, 14 y 15* abordan el diseño polimórfico, contratos mediante interfaces y su representación formal en UML:

• **Patrón Strategy (ITarifa):** En lugar de evaluar cadenas de texto o códigos mediante estructuras condicionales (if (modalidad == "PESO") ...) que violan el Principio Abierto/Cerrado (OCP), la liquidación financiera se delega en la interfaz ITarifa con sus implementaciones concretas: TarifaFija, TarifaPorPeso y TarifaPorDistancia. Cada clase encapsula su propia matemática. **Evidencia de control:** El despacho NX-1001 de Comercio Caribe (peso total 3.7 kg y tarifa de $3.500/kg) liquida exactamente **$12.950**, demostrando la precisión del polimorfismo.

• **Patrón Factory Method (CreadorLectores):** Centraliza la instanciación de los lectores especializados según la extensión del archivo (.json o .xml). La capa de aplicación depende únicamente del contrato ILectorArchivo, desconociendo los pormenores de serialización de cada formato.

• **Exportación Polimórfica (IExportadorInforme):** Implementada mediante ExportadorJson y ExportadorXml. Ambos garantizan de forma innegociable la codificación explícita Encoding.UTF8 (exigencia de cátedra), serializando los informes operativo, financiero y de inconsistencias sin dependencias cruzadas.

• **Modelado UML Limpio (Draw.io - *Sesión 15*):** En el diagrama actualizado: **(1)** Se incorporó el bloque visual de persistencia y exportación, **(2)** Se corrigió la relación de composición entre Despacho y Acontecimiento, orientando el rombo negro hacia Despacho (vida dependiente), **(3)** Se eliminaron factorías vacías no fundamentadas y la clase redundante AcuerdoComercial, y **(4)** Se respetó la prohibición de asignar valores a constantes en UML (cero = 34) y cero etiquetas redundantes {override}.

## 5. Análisis Técnico de Decisiones Clave y Erradicación de Antipatrones

### 5.1. Traslado del Mapeo a los DTOs (Principio de Experto en Información)

El patrón GRASP **Information Expert** (*Sesión 03, p. 19*) establece que la responsabilidad de procesar o transformar una estructura de datos debe asignarse a la clase que posee dicha información. Anteriormente, LectorPedidosJson y Empresa inspeccionaban los campos de PedidoJsonDto para ensamblar objetos de dominio. Al incorporar MapToDomain() dentro de PedidoJsonDto, AcuerdoDto, PaqueteJsonDto y AcontecimientoJsonDto, logramos que los DTOs asuman su rol de frontera, traduciendo datos crudos a entidades válidas y blindando al núcleo de dominio de etiquetas de serialización ([JsonPropertyName]).

### 5.2. Erradicación del 'Modelo de Dominio Anémico' en las Clases de Informe

Martin Fowler define el antipatrón *Anemic Domain Model* como aquel donde las clases solo contienen datos (getters y setters) mientras que la lógica y los cálculos se concentran en un servicio externo gigante. En el diseño anterior, InformeOperativo e InformeFinanciero eran clases vacías, mientras que Empresa ejecutaba bucles de 230 líneas para calcular promedios y sumas. En la refactorización, incorporamos los métodos estáticos InformeOperativo.Crear(...), InformeFinanciero.Crear(...) e InformeInconsistencias.Crear(...). Cada informe consolida sus propias métricas al recibir la colección de despachos y comercios. Empresa se limita a delegar: return InformeOperativo.Crear(_despachosPorGuia.Values, _establecimientos.Values);.

### 5.3. Regla de Negocio Crítica del SLA (Service Level Agreement)

Por directriz explícita del enunciado y la memoria operativa del proyecto: **El porcentaje de SLA se evalúa exclusivamente sobre el universo de despachos entregados**:

**% SLA = (Despachos Entregados a Tiempo / Total Despachos Entregados) * 100**

Los despachos con estado final *Devuelto*, *Pendiente* o *En Distribución* representan operaciones no concluidas o fallidas que se reportan en el informe operativo pero **no forman parte del denominador del indicador de puntualidad**. En el conjunto de datos de prueba: 6 despachos entregados (4 a tiempo, 2 tardíos) generan un cumplimiento exacto del **66.67%**.

### 5.4. Resiliencia Operativa ante Archivos Heterogéneos y Corruptos

El proyecto final simula la ingesta real de 5 archivos de datos con inconsistencias intencionadas. Un sistema frágil caería con excepciones no controladas. NexoLog implementa una estrategia de **resiliencia transaccional**: cuando se detecta una anomalía (guías duplicadas, identificadores de paquetes repetidos, pesos <= 0, referencias inexistentes o secuencias de seguimiento incompatibles), el sistema captura la inconsistencia, crea un RegistroInconsistencia con fecha, archivo origen y causa, y **continúa procesando las operaciones sanas**. El sistema detecta exactamente **17 inconsistencias controladas** sin abortar la ejecución.

### 5.5. Prohibición Innegociable de 'readonly' y su Equivalente Académico

Una de las reglas más estrictas del docente es la **prohibición total de la palabra clave readonly** (tanto en campos private readonly como en colecciones IReadOnlyList o métodos .AsReadOnly()). **Fundamento de Cátedra:** El profesor evalúa la comprensión de los mecanismos fundamentales de POO pura. En la cátedra, la inmutabilidad y la protección de solo lectura se modelan exclusivamente mediante: **(1)** Campos privados convencionales (private List _paquetes;), **(2)** Propiedades con solo get (public List Paquetes { get { return _paquetes; } }), y **(3)** Métodos de dominio para agregar elementos (AgregarPaquete(Paquete)). La refactorización cumple esta norma al 100% en todos sus archivos.

## 6. Guía Táctica: Banco de Preguntas y Respuestas para la Sustentación Oral

Compañero: durante la sustentación, el profesor evaluará no solo que el código funcione, sino la **capacidad de argumentar cada decisión técnica**. A continuación tienes las respuestas directas, estructuradas y conceptuales ante las preguntas más probables:

**Pregunta 1: ¿Por qué mantuvieron la clase Empresa y qué responsabilidades le quedaron?**

**Respuesta Defensiva:** *'Profesor, en la Sesión 03 aprendimos que las clases deben representar sustantivos tangibles del dominio del problema y no artificios técnicos como SistemaLogistico. Empresa es la entidad raíz de nuestro modelo logístico. En la refactorización eliminamos el antipatrón God Object despojándola de responsabilidades ajenas como lectura de archivos en disco, parsing de DTOs y bucles de reportes. Hoy Empresa se enfoca en su responsabilidad CRC pura: coordinar el diccionario de establecimientos, el índice de despachos y delegar los acontecimientos de seguimiento hacia la máquina de estados de cada despacho.'*

**Pregunta 2: ¿Por qué el mapeo DTO -> Dominio está en los DTOs y no en los lectores o en Empresa?**

**Respuesta Defensiva:** *'Aplicamos el patrón GRASP de Experto en Información (Information Expert) visto en las Sesiones 03 y 05. El DTO es la frontera tecnológica que conoce los nombres de propiedades del JSON o XML. Si colocáramos el mapeo en Empresa o en el dominio, contaminaríamos nuestras entidades de negocio con detalles del formato de transporte. Al ubicar MapToDomain() dentro del DTO, el DTO se responsabiliza de validar su propia estructura y construir una entidad de dominio válida, permitiendo que los lectores simplemente lean y deleguen.'*

**Pregunta 3: ¿Por qué los reportes calculan sus propias métricas en lugar de calcularlos en Empresa?**

**Respuesta Defensiva:** *'Para erradicar el antipatrón de Modelo de Dominio Anémico. En las Sesiones 03 y 05 vimos que un objeto en POO no es una simple estructura pasiva con getters y setters al estilo de lenguaje C, sino una entidad que encapsula datos y comportamiento. Antes, Empresa tenía más de 230 líneas calculando métricas para poblar clases huecas. Al implementar InformeOperativo.Crear(...) e InformeFinanciero.Crear(...), los reportes asumen la consolidación de sus indicadores, logrando una alta cohesión y liberando a Empresa de lógica de agregación externa.'*

**Pregunta 4: ¿Para qué sirve un constructor privado sin parámetros (private MiClase() { })?**

**Respuesta Defensiva:** *'Representa el balance perfecto entre el rigor de encapsulamiento exigido en la Sesión 05 y las restricciones de la plataforma .NET. Los motores de serialización y reflexión de C# exigen un constructor sin parámetros para instanciar tipos al deserializar. Si creáramos un constructor público sin parámetros, violaríamos la regla de que ningún objeto debe nacer en un estado inválido o nulo. Al declararlo privado, el código cliente está obligado a usar el constructor parametrizado con todas sus validaciones de negocio, mientras que la reflexión interna de .NET puede reconstruir el grafo de objetos de manera segura.'*

**Pregunta 5: ¿Cómo funciona el patrón Strategy en las tarifas y cómo garantiza extensibilidad?**

**Respuesta Defensiva:** *'Basándonos en la Sesión 12 (Polimorfismo) y Sesión 14 (Interfaces), encapsulamos la política de liquidación económica bajo el contrato ITarifa. Cada política comercial (TarifaFija, TarifaPorPeso, TarifaPorDistancia) implementa su fórmula en su método CalcularTarifa(despacho). Esto elimina cadenas de switch o if/else y cumple el Principio Abierto/Cerrado (OCP): si mañana NexoLog crea una tarifa por volumen, solo creamos una nueva clase sin tocar Despacho ni Empresa. Prueba de ello es que en la guía NX-1001 de Comercio Caribe, la tarifa por peso calculó exactamente $12.950.'*

**Pregunta 6: ¿Por qué la fórmula de SLA solo divide entre despachos entregados y no entre el total?**

**Respuesta Defensiva:** *'Porque el Service Level Agreement evalúa la puntualidad de los envíos que concluyeron su ciclo operativo de entrega. Un despacho devuelto por dirección errónea o un despacho que aún se encuentra en ruta no pueden calificarse como entregas a tiempo ni como entregas tardías; corresponden a excepciones operativas o estados transitorios. Por tanto, el denominador estricto de la métrica es el conjunto de despachos con estado Entregado. En nuestras pruebas, de 6 entregados, 4 fueron a tiempo y 2 fuera de plazo, arrojando un 66.67% de cumplimiento exacto.'*

**Pregunta 7: ¿Por qué fue necesario ordenar cronológicamente los acontecimientos de seguimiento?**

**Respuesta Defensiva:** *'Los archivos XML de las transportadoras presentan acontecimientos en desorden temporal (por ejemplo, el evento de entrega registrado antes de la salida a ruta). La máquina de estados de Despacho valida invariantes estrictos: un paquete no puede ser entregado si no ha sido previamente despachado. Si procesáramos los hitos en el orden crudo del archivo, la validación lanzaría una excepción y rechazaría una entrega legítima. Por eso, en Empresa.ProcesarEventosDeGuia ordenamos la colección por FechaHora de forma ascendente antes de invocar la máquina.'*

**Pregunta 8: ¿Por qué no usaron la palabra clave 'readonly' en ninguna clase?**

**Respuesta Defensiva:** *'Por respeto estricto a las normas de cátedra acordadas. En la asignatura de POO se nos enseña a modelar el encapsulamiento y la protección de datos mediante los principios clásicos de orientación a objetos: campos privados inicializados en el constructor y expuestos exclusivamente a través de propiedades con solo get. El uso de readonly o IReadOnlyList oculta el entendimiento conceptual de cómo operan las referencias y el encapsulamiento a nivel de lenguaje.'*

## 7. Protocolo de Verificación y Compilación (.NET 10)

La solución ha sido verificada empíricamente de extremo a extremo en el entorno de desarrollo oficial (.NET 10 SDK 10.0.401). A continuación se detallan los comandos de compilación y los resultados de control obtenidos:

**COMANDOS DE CONSOLA VERIFICADOS: 1. Compilación Limpia:**
dotnet build 04_Proyecto_Final/NexoLog/NexoLog.csproj
**Resultado:** Compilación correcta. 0 Advertencias, 0 Errores.

**2. Ejecución Automatizada y Control de Salida:**
dotnet run --project 04_Proyecto_Final/NexoLog/NexoLog.csproj
**Artefactos Generados:**
• InformesExportados/informes_nexolog.json (UTF-8, 9.4 KB)
• InformesExportados/informes_nexolog.xml (UTF-8, 9.8 KB)

| Métrica de Control | Valor Esperado | Valor Obtenido | Estado |
|---|---|---|---|
| Despachos Únicos Consolidados | 11 despachos | 11 despachos | **CORRECTO** |
| Estados Finales | 6 Entr, 2 Dev, 1 Dist, 2 Pend | 6 Entr, 2 Dev, 1 Dist, 2 Pend | **CORRECTO** |
| Evaluación SLA sobre Entregados | 4 / 6 = 66.67% | 4 / 6 = 66.67% | **CORRECTO** |
| Prueba de Control Guía NX-1001 | $12.950 (3.7 kg * $3.500) | $12.950 | **CORRECTO** |
| Gran Total Facturado | $104.450 | $104.450 | **CORRECTO** |
| Inconsistencias Manejadas | 17 inconsistencias | 17 inconsistencias | **CORRECTO** |

**Conclusión y Recomendación para la Sustentación:** Compañero, contamos con una arquitectura impecable, alineada al 100% con la visión de la cátedra del profesor Paternina. La reducción de Empresa.cs a dominio puro, el mapeo en los DTOs, la autonomía de los reportes y los constructores privados representan decisiones de ingeniería maduras. Estudia detenidamente este documento y el banco de preguntas; tenemos todos los argumentos técnicos para alcanzar la máxima calificación.